Non-Linear Iterative Solvers

Jonathan Cushman

February 2, 2026

1 Executive Summary

The following report discusses the design, implementation, and validation for distinct nonlinear solvers. Included in the report is a mathematical breakdown of the algorithms that were implemented, the various design aspects that were chosen throughout that process, and the validation of the iterative methods through empirical observations and evidence based claims.

2 Statement of the Problem

The general task for this assignment is to implement a selection of iterative solvers and demonstrate how they behave given certain initial conditions. The methods being analyzed are: Newton's Method, Modified Newton's Method, Steffensen's Method, Regula Falsi, and the Secant Method. These methods will be tested against higher order roots, distinct roots, scaled distinct roots, and roots that coalesce. In addition to testing the various methods, an implementation of the solutions for questions 6.3 and 6.10 are included to demonstrate nonlinear relaxation iterations for systems and show confirmation of the analysis.

3 Description of the Mathematics

The problem is to find $x^* \in \mathbb{R}$ such that

$$f(x^*) = 0 \tag{1}$$

where $f(x)$ may be nonlinear and have many solutions. To find the root of the nonlinear equation we can instead find the roots to a series of linear functions $l_k(x)$ and approximate $f(x)$ with a line in the neighborhood of $x^{(k)}$.

$$l_k(x) = f(x^{(k)}) + (x - x^{(k)})q_k \tag{2}$$

Where q_k defines the method of choice. For Newtons Method define:

$$q_k = f'(x^{(k)}) \rightarrow x^{(k+1)} = x^{(k)} - (m) * q_k^{-1} f(x^{(k)}) \tag{3}$$

As seen, Newton's Method requires only one initial guess in order to produce a new sequence that locally converges to a root. It is generally known to have a quadratic convergence rate along with knowledge about the Derivative. Also consider the variable $m \in \mathbb{Z}^+$ which is used to define Modified Newton's Method when $m > 1$

The next method to Define is Steffensen's Method:

$$q_k = \theta(x^{(k)}) = \frac{f(x^{(x)} + f(x^{(k)})) - f(x^{(k)})}{f(x^{(x)})} \rightarrow x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{\theta(x^{(k)})} \tag{4}$$

Similar to Newton, Steffensen's Method only requires one initial guess. The difference comes in the fact that the derivative is replaced with the difference approximation. This can numerically simulate the derivative by a finite difference approximation.

The following methods are defined to have two initial points in order to create an iteration. First the Secant Method:

$$q_k = \frac{f(x^k) - f(x^{(k-1)})}{x^{(k)} - x^{(k-1)}} \rightarrow x^{(k+1)} = x^{(k)} - q_k^{-1} f(x^{(k)}) \tag{5}$$

The Secant Method is a locally convergent method which means the initial guess must be close enough in order for convergence to occur. Additionally this method does not stay within the intervals which were previously defined by the iteration. It is also known to have a super-linear convergence rate.

The last iterative method to define is Regula Falsi:

$$q_k = \frac{(b^{(k)}) - f(a^{(k)})}{b^{(k)} - a^{(k)}} \rightarrow x^{(k+1)} = x^{(k)} - q_k^{-1} f(x^{(k)}) \tag{6}$$

where $[a^{(k)}, b^{(k)}]$ defines the initial interval. This method differs from the rest in the fact that it must satisfy the condition $f(a^{(k)})f(b^{(k)}) < 0$ at each step. This ensure the existence of a root by the Intermediate Value Theorem.

4 Implementations of The Algorithms

The implementation of each algorithm follows a similar structure and pattern, which follows from the class notes.

The first algorithm to discuss is Newton's Method. This method begins with an initial guess, which is then sent into a loop which counts the number of iterations that will be used until a root is found. Inside this loop includes an evaluation of the function and it's derivative. The derivative is then checked

with a conditional to ensure it is not below machine precision. If this were to happen then the iteration would include a division by zero and thus unable to divide by the derivative. If this conditional is passed then the next updated value of x is created, another iteration is added, and the progress is stored in an dynamic array. The last section check if the new value created is a root below the threshold of machine precision and if we have found a root the function return the updated x value. otherwise the process is repeated until a root is found or the preset number of test trials expires.

Listing 1: Algorithm

```
function [x, list, iterations] = Newtons_Method(f,df,x_guess,m,num_trials)
% Things being returned
    % x - a root of the function (hopefully)
    % list - tracks the progress of x
    % iterations - tracks how long did it take to converge

% Notes to self: may want to lower the tolerance from eps
% Convergence should be quadratic

% f - function
% df - derivative

% Input Initial Guess
x = x_guess;
% Array to track x throughout the iterations
list = x_guess;
iterations = 0;

% start iterations
while iterations < num_trials
    % evaluate the function
    fx = f(x);
    dfx = df(x); % Evaluate the derivative at the current x

    % check if the derivative to too close to 0 to divide
    if abs(dfx) < eps
        fprintf("Derivative is approx zero, unable to divide \n");
       break;
    end

    % NEWTONS METHOD (modified if m > 1)
    x_new = x - m * (fx/dfx);
    iterations = iterations + 1;
    % Track the progress of x
    list = [list,x_new];
```

```
    % Stopping Critera
    if abs(f(x_new)) < eps
        % Root found
        x = x_new;
        break;
    end
    % update the new iteration
    x = x_new;
end
end
```

The next algorithm to examine is Steffensen's Method. The main difference between this methods algorithm and Newton's is the evaluation of the difference approximation $\theta(x)$ rather than the derivative $f'(x)$. A similar check is then used to ensure division by zero does not occur within our update. Then create the updated iteration and check if the update created a root, otherwise repeat this until convergence or the number of trials reaches the preset limit.

Listing 2: Algorithm

```
function [x, list, iterations] = Steffensons_Method(f, x_guess, num_trials)

% Follows Same style as Newtons Method
% Replaces the derivative with difference approximation: Theta
% Notation: x is x_k and x_new is x_k+1

% How fast does this converge - Quadratic?

x = x_guess;
list = x_guess;
iterations = 0;

while iterations < num_trials
    fx = f(x);
    % evaluate Theta
    thetax = (f(x + f(x)) - f(x))/ f(x);

    if abs(thetax) < eps
        print("Division by Zero Error in Theta");
        break;
    end

    %evaluate next iteration
    x_new = x - fx/thetax;
    iterations = iterations + 1;
```

```
    list = [list, x_new];

    if abs(f(x_new)) < eps
        x = x_new;
        break;
    end

    % update old iteration
    x = x_new;
end
end
```

Looking towards the next algorithm, the Secant Method follows the same pattern as the previous methods, but differs in the fact that two initial guesses are need to create the updated value. Similar to before, the main convergence loop begins with function evaluations which are then used to find the value of q_k, which is subsequently checked to not be below machine precision and avoid division by zero. Then the new update is created and added to the iteration progress, followed by the termination criteria. If a root is not found then the two initial points are updated.

Listing 3: Algorithm

```
function [x2, list, iterations] = Secant_Method(f,x_1,x_2,num_trials)
% Different from Newton and Steffenson Method
% Need 2 initial guesses to be close to interval for conv.
% "Superlinear Q-Convergence Rate"

%x1 = x^k-1
x1 = x_guess1;
%x = x^k
x2 = x_guess2;

%start the progress with both inital points
list = [x1, x2];
iterations = 0;

while iterations < num_trials
    fx1 = f(x1);
    fx2 = f(x2);
    % compute q for iterations AKA slope
    q = (fx2 - fx1) / (x2 - x1);

    if abs(q) < eps
        fprintf("difference in iterations is approx zero \n");
       break;
```

```
    end

    % Update new value of x in iteration
    x_new =  x2 - (fx2/q);
    iterations = iterations + 1;
    list = [list , x_new];

    % Check for root
    if abs(f(x_new)) < eps
        x2 = x_new;
        break; %root has been found
    end

    % Otherwise a root has not been found yet
    % Update previous iterations to next iterations
    x1 = x2;
    x2 = x_new;
end
end
```

The final algorithm is to be evaluated is the method of Regula Falsi. This method is similar to that of the Secant Method where is requires two initial points, but has the unique trait that $f(a)f(b) < 0$ in order to produce a sequence of shrinking intervals. Without the confirmation of this step we can guarantee the existence of a root. This is why our function is evaluated before the main convergence loop in order to provide this condition. After checking the initial interval the iteration begins and a check is performed for convergence. An additional check is something to be noted for this method where if the distance between a and b is less than machine precision we have almost surely arrived at a root. If the convergence check is not satisfied then we update the size of the interval by replacing a or b depending upon which evaluations continue to satisfy the condition $f(a)f(b) < 0$ until the termination criteria hits or the number of preset trials runs out.

Listing 4: Algorithm

```
function [x_new,list,iterations] = RegulaFalsi_Method(f, a, b, num_trials)
% [a,b] - inital interval
% f(a)*f(b) < 0 to ensure existence of a root
% Convergence should be linear
a= a_guess;
b= b_guess;

fa = f(a);
fb = f(b);

if fa * fb > 0
```

```
    fprintf("Unuseable interval \n")
end

% start with a as that will be the first x value we use
list = a;
iterations = 0; % Initialize iteration counter

% begin RF Method
while iterations < num_trials
    q = (fb - fa)/ (b-a);
    % x can be a or b
    x = a;
    % Update iteration
    x_new = x - f(x)/q;

    iterations = iterations + 1;
    list = [list , x_new];
    % evaluate new x value
    f_new = f(x_new);

    % Check for Convergence
    if (abs(f_new) < eps) || ((b-a) < eps)
        break;
    end

    % Update interval based on the sign of f_new
    if fa * f_new < 0
        b = x_new;
        fb = f_new;
    else
        a = x_new;
        fa = f_new;
    end
end
%return x_new as it could be a or b
end
```

5 Experimental Design and Results

To empirically evaluate the algorithms and identify their strengths and weaknesses, a selection of tasks have been evaluated to identify potential shortcomings and edge cases from the algorithms. A brief synopsis includes a function with higher order roots, a function with three distinct roots, and a function where roots coalesce. In addition to this, some nonlinear relaxation methods will be analyzed as well to confirm the results derived from the analysis of the

iterations and how this generalizes to systems.

5.1 Task 1: Higher Order Roots

For this task, consider the function $f(x) = (x - \rho)^d$ where ρ is a single root of multiplicity d. For the entirety this task let $\rho = 0$.

The first method to consider is Standard Newton's Method ($m = 1$) where the values of d are increasing. For these tests, initial guesses of 1 and 10 were used to verify the method. The results indicate that as the multiplicity of the root increases, more iteration are needed to reach the root which is to be expected. This comes from the fact that the steepness of the function being less intense as d increase which causes the step size of the iteration to decrease slightly each time, results in the slower convergence.

Figure 1: Initial Guess: $x_0 = 1$

```
 Standard Newton Method (m=1, d=2,3,...)
d= 2 | Iterations: 27 | Root: 0.000000
d= 3 | Iterations: 30 | Root: 0.000005
d= 4 | Iterations: 32 | Root: 0.000100
d= 5 | Iterations: 33 | Root: 0.000634
d= 6 | Iterations: 33 | Root: 0.002438
d= 7 | Iterations: 34 | Root: 0.005294
d= 8 | Iterations: 34 | Root: 0.010673
d= 9 | Iterations: 35 | Root: 0.016205
d= 10 | Iterations: 35 | Root: 0.025032
d= 11 | Iterations: 35 | Root: 0.035584
d= 12 | Iterations: 35 | Root: 0.047577
d= 13 | Iterations: 35 | Root: 0.060719
d= 14 | Iterations: 35 | Root: 0.074737
d= 15 | Iterations: 35 | Root: 0.089390
d= 16 | Iterations: 35 | Root: 0.104471
d= 17 | Iterations: 35 | Root: 0.119808
d= 18 | Iterations: 36 | Root: 0.127747
d= 19 | Iterations: 36 | Root: 0.142784
d= 20 | Iterations: 36 | Root: 0.157779
```

considering this same test for modified newtons method where ($m = d = 2, 3, ...$). In this scenario the the multiplicity of the root is equal to the scalar of the slope in our iteration. This results in a one step instantaneous convergence for all initial point tested at 1, 10, and 100.

Figure 2: Initial Guess: $x_0 = 1$

```
 Modified Newton Method (m=d=2,3,...)
d= 2  | Iterations: 1  | Root: 0.000000
d= 3  | Iterations: 1  | Root: 0.000000
d= 4  | Iterations: 1  | Root: 0.000000
d= 5  | Iterations: 1  | Root: 0.000000
d= 6  | Iterations: 1  | Root: 0.000000
d= 7  | Iterations: 1  | Root: 0.000000
d= 8  | Iterations: 1  | Root: 0.000000
d= 9  | Iterations: 1  | Root: 0.000000
d= 10  | Iterations: 1  | Root: 0.000000
d= 11  | Iterations: 1  | Root: 0.000000
d= 12  | Iterations: 1  | Root: 0.000000
d= 13  | Iterations: 1  | Root: 0.000000
d= 14  | Iterations: 1  | Root: 0.000000
d= 15  | Iterations: 1  | Root: 0.000000
d= 16  | Iterations: 1  | Root: 0.000000
d= 17  | Iterations: 1  | Root: 0.000000
d= 18  | Iterations: 1  | Root: 0.000000
d= 19  | Iterations: 1  | Root: 0.000000
d= 20  | Iterations: 1  | Root: 0.000000
```

Running Steffensen's Method next we evaluate the convergence rate as it compares to Newton's. Looking at the test results, Steffensen's method appear to be quadratic and similar to that of Newtons for small values of d, but as d grows, the convergence rate slows down immensely and is not longer able to compete with newtons method. Also notice that at no point in time was Newton's Method "beat" by Steffensen's. We can associate this with the fact that Steffensen's Method illustrate a finite difference method which is discritizing the point rather than taking the instant rate of change resulting in slower convergence and significant drop offs in the rate as d grows.

Figure 3: Initial Guess: $x_0 = 1$

```
 Steffensons Method (d = 2,3,...)
d= 2  | Iterations: 28  | Root: 0.000000
d= 3  | Iterations: 32  | Root: 0.000006
d= 4  | Iterations: 35  | Root: 0.000116
d= 5  | Iterations: 38  | Root: 0.000659
d= 6  | Iterations: 41  | Root: 0.002238
d= 7  | Iterations: 45  | Root: 0.005248
d= 8  | Iterations: 50  | Root: 0.010758
d= 9  | Iterations: 58  | Root: 0.018141
d= 10  | Iterations: 71  | Root: 0.026323
d= 11  | Iterations: 92  | Root: 0.034862
d= 12  | Iterations: 125  | Root: 0.049183
d= 13  | Iterations: 182  | Root: 0.060057
d= 14  | Iterations: 278  | Root: 0.075791
d= 15  | Iterations: 445  | Root: 0.085682
d= 16  | Iterations: 734  | Root: 0.104970
d= 17  | Iterations: 1000  | Root: 0.982095
d= 18  | Iterations: 1000  | Root: 0.994679
d= 19  | Iterations: 1000  | Root: 0.997751
d= 20  | Iterations: 1000  | Root: 0.998960
```

Next, Looking at Regula Falsi for $d = 2, 3, ..., n$ a unique pattern begins to appear. When d is an odd integer the iteration converges instantaneously, while for even powers the iteration diverges. This can be derived from the initial Theorem that must be satisfied where $f(a)f(b) < 0$. Looking at the graph of $f(x) = (x - \rho)^d$ it is easy noticed that the multiplicity reflects the graph on the y-axis for $x < 0$, when the power is even, a parabola is formed and thus our initial condition is never satisfied. Another point to add is that when our initial guesses are symmetric $[-a, a]$ our iteration converges instantly. but when the initial interval is not mirrored, the convergence rate slows down and more iteration are needed to reach the desired root.

Figure 4: Initial Interval: $[-1, 1]$

```
 Regula Falsi (d = 1,2,3,...)
d= 1  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 2  | Iterations: 1000  | Root: NaN
d= 3  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 4  | Iterations: 1000  | Root: NaN
d= 5  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 6  | Iterations: 1000  | Root: NaN
d= 7  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 8  | Iterations: 1000  | Root: NaN
d= 9  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 10  | Iterations: 1000  | Root: NaN
d= 11  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 12  | Iterations: 1000  | Root: NaN
d= 13  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 14  | Iterations: 1000  | Root: NaN
d= 15  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 16  | Iterations: 1000  | Root: NaN
d= 17  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 18  | Iterations: 1000  | Root: NaN
d= 19  | Iterations: 1  | Root: 0.000000
Unuseable interval
d= 20  | Iterations: 1000  | Root: NaN
```

Looking towards the Secant Method the results indicate the method is reliably convergent for initial guesses that are close to the root and do not mirror each other. Similarly to Regula Falsi when our initial guesses mirror each other ($x_0 = -x_1$) we are unable to use the method when d is odd. This causes a division by zero in the calculation for q_k^{-1} while even powers of d converge instantly from the functions symmetry. On the other hand, when the initial guesses do not mirror each other the iteration works quite well. One more noticeable point is that when $d \geq 24$ the number of iterations explodes from a reasonable 52 to hitting the limit of 1000. Once again this can be attributed to the steepness of the function increasing as the multiplicity grows. This leads to a much smaller relative step size and therefore the slower convergence

Figure 5: Initial Guesses: $x_0 = -1$ and $x_1 = 1$

```
 Secant Method (d = 1,2,3,...)
d= 1  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 2  | Iterations: 0  | Root: 1.000000
d= 3  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 4  | Iterations: 0  | Root: 1.000000
d= 5  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 6  | Iterations: 0  | Root: 1.000000
d= 7  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 8  | Iterations: 0  | Root: 1.000000
d= 9  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 10  | Iterations: 0  | Root: 1.000000
d= 11  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 12  | Iterations: 0  | Root: 1.000000
d= 13  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 14  | Iterations: 0  | Root: 1.000000
d= 15  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 16  | Iterations: 0  | Root: 1.000000
d= 17  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 18  | Iterations: 0  | Root: 1.000000
d= 19  | Iterations: 1  | Root: 0.000000
difference in iterations is approx zero, unable to divide
d= 20  | Iterations: 0  | Root: 1.000000
```

Next consider the case of Modified Newton's Method for ($d \geq 2$) where ($m \neq d$). when $m < d$ the method converges to the root for all the initial guesses 1 and 10. As the value of $m \to d$ the convergence rate speed up much quicker. This differed drastically from the case where $m > d$ and we see convergence on extremely rare cases. when d is large and m is only slightly greater then d that we see convergence. as m and d converge closer and closer to one

another we see the convergence speed up dramatically until $(m = d)$ which when observed earlier had one step convergence.

Figure 6: Initial Guesses: $x_0 = -1$ and $x_1 = 1$

```
 Modified Newton Method (m < d)
m = 1, d= 2  | Iterations: 27  | Root: 0.000000
m = 1, d= 3  | Iterations: 30  | Root: 0.000005
m = 2, d= 3  | Iterations: 11  | Root: 0.000006
m = 1, d= 4  | Iterations: 32  | Root: 0.000100
m = 2, d= 4  | Iterations: 14  | Root: 0.000061
m = 3, d= 4  | Iterations: 7  | Root: 0.000061
m = 1, d= 5  | Iterations: 33  | Root: 0.000634
m = 2, d= 5  | Iterations: 15  | Root: 0.000470
m = 3, d= 5  | Iterations: 8  | Root: 0.000655
m = 4, d= 5  | Iterations: 5  | Root: 0.000320
m = 1, d= 6  | Iterations: 33  | Root: 0.002438
m = 2, d= 6  | Iterations: 15  | Root: 0.002284
m = 3, d= 6  | Iterations: 9  | Root: 0.001953
m = 4, d= 6  | Iterations: 6  | Root: 0.001372
m = 5, d= 6  | Iterations: 4  | Root: 0.000772
m = 1, d= 7  | Iterations: 34  | Root: 0.005294
m = 2, d= 7  | Iterations: 16  | Root: 0.004591
m = 3, d= 7  | Iterations: 10  | Root: 0.003712
m = 4, d= 7  | Iterations: 7  | Root: 0.002656
m = 5, d= 7  | Iterations: 5  | Root: 0.001904
m = 6, d= 7  | Iterations: 3  | Root: 0.002915
m = 1, d= 8  | Iterations: 34  | Root: 0.010673
m = 2, d= 8  | Iterations: 16  | Root: 0.010023
m = 3, d= 8  | Iterations: 10  | Root: 0.009095
m = 4, d= 8  | Iterations: 7  | Root: 0.007812
m = 5, d= 8  | Iterations: 5  | Root: 0.007416
m = 6, d= 8  | Iterations: 4  | Root: 0.003906
m = 7, d= 8  | Iterations: 3  | Root: 0.001953
```

The last consideration for this task is to evaluate all the methods when $d = 1$. This leads to the evaluation of the function $f(x) = x - \rho$. This is a linear function and each method we are evaluating uses a linear local model. Using any method therefore will produce a perfect slope that mirrors the function we are evaluating. This leads to a one step convergence for each method as the first iteration will mirror the function perfectly and leads directly to the desired root.

Figure 7: Initial Guesses: $x_0 = -1$ and $x_1 = 1$

```
 Modified Newton Method (m > d)
m = 2, d= 1  | Iterations: 1000  | Root: 1.000000
m = 3, d= 1  | Iterations: 1000  | Root: 10715086071
m = 3, d= 2  | Iterations: 27  | Root: -0.000000
m = 4, d= 1  | Iterations: 1000  | Root: NaN
m = 4, d= 2  | Iterations: 1000  | Root: 1.000000
m = 4, d= 3  | Iterations: 11  | Root: -0.000006
m = 5, d= 1  | Iterations: 1000  | Root: NaN
m = 5, d= 2  | Iterations: 1000  | Root: NaN
m = 5, d= 3  | Iterations: 30  | Root: 0.000005
m = 5, d= 4  | Iterations: 7  | Root: -0.000061
m = 6, d= 1  | Iterations: 1000  | Root: NaN
m = 6, d= 2  | Iterations: 1000  | Root: NaN
m = 6, d= 3  | Iterations: 1000  | Root: 1.000000
m = 6, d= 4  | Iterations: 14  | Root: 0.000061
m = 6, d= 5  | Iterations: 5  | Root: -0.000320
m = 7, d= 1  | Iterations: 1000  | Root: NaN
m = 7, d= 2  | Iterations: 1000  | Root: NaN
m = 7, d= 3  | Iterations: 1000  | Root: NaN
m = 7, d= 4  | Iterations: 32  | Root: 0.000100
m = 7, d= 5  | Iterations: 8  | Root: 0.000655
m = 7, d= 6  | Iterations: 4  | Root: 0.000772
m = 8, d= 1  | Iterations: 1000  | Root: NaN
m = 8, d= 2  | Iterations: 1000  | Root: NaN
m = 8, d= 3  | Iterations: 1000  | Root: NaN
m = 8, d= 4  | Iterations: 1000  | Root: 1.000000
```

5.2 Task 2: Distinct Roots and Coalescing Trends

5.2.1 Three Distinct Roots

For the following tests consider the function: $f(x) = x(x-\rho)(x+\rho)$ where $\rho > 0$ and it's derivative: $f'(x) = 3(x-\alpha)(x+\alpha)$ where $\alpha = \frac{\rho}{\sqrt{3}} > 0$. For the following test let $\rho = 2$

Testing Newton's Method resulted in convergence for $x > \rho$, $\alpha < x < \rho$, and $\xi < x < \xi$. The first two scenario converge to the rightmost root $\rho = 2$ while the initial guess within the cycling point converged to the middle root at 0. The value of ξ can be derived from setting and solving the iteration for:

$$\phi(\xi) = \xi - \frac{f(\xi)}{f'(\xi)} = -\xi \rightarrow \xi = \frac{\rho}{\sqrt{5}} \tag{7}$$

Figure 8: Intervals of Interest

```
Newton behavior on the Interval: x > p
root: 2.000000 |  Iterations: 7
:     4.0000    2.9091    2.3021    2.0507    2.0018    2.0000    2.0000    2.0000

Newton behavior on the Interval: a < x < p
root: 2.000000 |  Iterations: 6
     1.5000    2.4545    2.1014    2.0069    2.0000    2.0000    2.0000

Newton behavior on the Interval: -c < x < c
root: -0.000000 |  Iterations: 4
    0.5000   -0.0769    0.0002   -0.0000   -0.0000

Newton behavior for x0 = c (cycling point)
root: 0.416750 |  Iterations: 20
  Columns 1 through 17

    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944
```

Given that $\alpha > \rho$ we see that somewhere in that neighborhood there is a point where the flow of information switches direction. When $x_0 = \xi$ newton's method does cycle for a while but eventually converges to the root at 0. Perturbing the cycling point such that $x_o = \xi + \epsilon$ had convergence to both roots 2 and -2 depending upon the size of the perturbation. For $x_o = \xi - \epsilon$, all iterations converge to 0.

Setting the initial guess $x_0 = \alpha$ caused problems for the algorithm. This forces a division by zero error when evaluating the derivative and the functions breaks away. Perturbing α also caused issues for Newton's Method. This is due to the fact that near alpha the derivative is close to zero which causes the iteration to move far away from its original value due to the almost constant slope being generated. Even with this the iteration was still able to converge to a root while initially being shot very far from the target.

Looking at Steffensen's Method next, for $x > \rho$ the method converged to the root at 2 at a much slower rate than Newton. When $\alpha < x < \rho$, $-\xi < x < \xi$, $x = \alpha$, and $x = \xi$ all iteration converged to the root at 0. Perturbing the initial

Figure 9: Cycle Point and Positive Perturbations

```
Newton behavior for x0 = c (cycling point)
root: 0.894427 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944

Newton Bahvior for Perturbed cycling point: x0 + eps
pertubation size: 0.10000000000 | root after perturbation: -2.000000 |  Iterations: 5
    0.9944   -1.9033   -2.0079   -2.0000   -2.0000   -2.0000

pertubation size: 0.01000000000 | root after perturbation: 2.000000 |  Iterations: 9
    0.9044   -0.9570    1.4001    2.9187    2.3068    2.0520    2.0019    2.0000    2.0000    2.0000

pertubation size: 0.00100000000 | root after perturbation: -15.120988 |  Iterations: 10
    0.8954   -0.9005    0.9315   -1.1573 -171.4361 -114.2959  -76.2051  -50.8150  -33.8942  -22.6224  -15.1210

pertubation size: 0.0001000000 | root after perturbation: -2.000000 |  Iterations: 10
    0.8945   -0.8950    0.8980   -0.9164    1.0397   -2.9691   -2.3321   -2.0597   -2.0025   -2.0000   -2.0000

pertubation size: 0.0000100000 | root after perturbation: 2.000000 |  Iterations: 10
    0.8944   -0.8945    0.8948   -0.8966    0.9075   -0.9776    1.6492    2.1567    2.0156    2.0002    2.0000

pertubation size: 0.0000010000 | root after perturbation: -2.737722 |  Iterations: 10
    0.8944   -0.8944    0.8945   -0.8946    0.8957   -0.9023    0.9430   -1.2587   -5.2985   -3.7084   -2.7377

pertubation size: 0.0000001000 | root after perturbation: -4.095362 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8946   -0.8952    0.8991   -0.9231    1.0897   -5.9084   -4.0954

pertubation size: 0.0000000100 | root after perturbation: 2.082658 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8945    0.8949   -0.8972    0.9115   -1.0045    2.0827

pertubation size: 0.0000000010 | root after perturbation: 0.958113 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8945   -0.8947    0.8961   -0.9046    0.9581

pertubation size: 0.0000000001 | root after perturbation: 0.900505 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8945    0.8946   -0.8954    0.9005
```

Figure 10: Cycle Point and Negative Perturbations

```
Newton Bahvior for Perturbed cycling point: x0 - eps
pertubation size: 0.10000000000 | root after perturbation: 0.000000 |  Iterations: 5
    0.7944   -0.4760    0.0650   -0.0001    0.0000         0

pertubation size: 0.01000000000 | root after perturbation: -0.000000 |  Iterations: 6
    0.8844   -0.8368    0.6172   -0.1646    0.0023   -0.0000   -0.0000

pertubation size: 0.00100000000 | root after perturbation: -0.000000 |  Iterations: 7
    0.8934   -0.8885    0.8595   -0.7117    0.2907   -0.0131    0.0000   -0.0000

pertubation size: 0.0001000000 | root after perturbation: 0.000000 |  Iterations: 9
    0.8943   -0.8938    0.8908   -0.8732    0.7776   -0.4302    0.0462   -0.0000    0.0000    0.0000

pertubation size: 0.0000100000 | root after perturbation: -0.000000 |  Iterations: 10
    0.8944   -0.8944    0.8941   -0.8923    0.8816   -0.8214    0.5611   -0.1156    0.0008   -0.0000   -0.0000

pertubation size: 0.0000010000 | root after perturbation: 0.000000 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8942    0.8931   -0.8867    0.8495   -0.6683    0.2244   -0.0059    0.0000

pertubation size: 0.0000001000 | root after perturbation: 0.025645 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8943   -0.8937    0.8898   -0.8671    0.7473   -0.3592    0.0256

pertubation size: 0.0000000100 | root after perturbation: 0.496918 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8943    0.8940   -0.8916    0.8779   -0.8015    0.4969

pertubation size: 0.0000000010 | root after perturbation: 0.836887 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8941    0.8927   -0.8844    0.8369

pertubation size: 0.0000000001 | root after perturbation: 0.888411 |  Iterations: 10
    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8944   -0.8944    0.8943   -0.8934    0.8884
```

Figure 11: Initial Guess = α

```
Newton behavior near alpha (slope approx 0): x0 move far away
Initial x0: 1.154702 | Root found: 11561.045067 | Iterations: 10
   1.0e+05 *

    0.0000    4.4445    2.9630    1.9753    1.3169    0.8779    0.5853    0.3902    0.2601    0.1734    0.1156

Newton behavior with x0 = alpha (Division by Zero):
Derivative is approx zero, unable to divide
Root: 1.154701 | Iterations: 0
    1.1547
```

value at the cycling point positively forced the iteration to converge to -2 at a slow rate while when perturbing through subtraction, all iteration go to 0 extremely fast.

Figure 12: Comparing Steffensen's Method

```
steffenson behavior on the Interval: a < x < p
root: 0.000000 |  Iterations: 5
     1.5000    0.2914   -0.0401    0.0001   -0.0000    0.0000

Steffenson behavior on the Interval: -c < x < c
root: 0.000000 |  Iterations: 5
    0.5000   -0.2362    0.0208   -0.0000    0.0000    0.0000

steffenson behavior for x0 = c (cycling point)
root: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000         0

steffenson behavior for x0 = alpha
root: 0.000000 |  Iterations: 5
    1.1547   -1.4434   -0.1107    0.0021   -0.0000    0.0000
```

Figure 13: Comparing Steffensen's Method for positive perturbations

```
Steffenson Bahvior for Perturbed cycling point: x0 + eps
pertubation size: 0.1000000000 | root after perturbation: -2.000000 |  Iterations: 4
    0.9944   -1.9992   -2.0000   -2.0000   -2.0000

pertubation size: 0.0100000000 | root after perturbation: 4.939485 |  Iterations: 10
    0.9044   -1.7795    5.0067    4.9985    4.9902    4.9818    4.9734    4.9650    4.9566

pertubation size: 0.0010000000 | root after perturbation: -0.000000 |  Iterations: 5
    0.8954   -1.7407    0.1857   -0.0099    0.0000   -0.0000

pertubation size: 0.0001000000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8945   -1.7367    0.0916   -0.0012    0.0000         0

pertubation size: 0.0000100000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7363    0.0830   -0.0009    0.0000         0

pertubation size: 0.0000010000 | root after perturbation: -0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0821   -0.0008    0.0000   -0.0000

pertubation size: 0.0000001000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000    0.0000

pertubation size: 0.0000000100 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000    0.0000

pertubation size: 0.0000000010 | root after perturbation: -0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000   -0.0000

pertubation size: 0.0000000001 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000    0.0000
```

Testing for Regula Falsi the Secant Method resulted in reliable convergence as well. For Regula Falsi it was expected that the initial intervals that satisfies the necessary condition would converge to the desired root, while intervals that did not satisfy the condition were issued a warning error and converged to the non target root. The Secant Method was also reliable in finding root. For situations when $x^{(0)} < \rho < x^{(1)}$ the method would converge to the target root and setting the initial points to α and ξ had no affect on the convergence.

Figure 14: Comparing Steffensen's Method for Negative perturbations

```
steffenson Bahvior for Perturbed cycling point: x0 - eps
pertubation size: 0.1000000000 | root after perturbation: -2.000000 |  Iterations: 8
    0.7944   -1.2295    0.9721   -1.9793   -2.0032   -2.0001   -2.0000   -2.0000   -2.0000

pertubation size: 0.0100000000 | root after perturbation: 0.000000 |  Iterations: 6
    0.8844   -1.6906   -0.3949    0.1067   -0.0018    0.0000         0

pertubation size: 0.0010000000 | root after perturbation: -0.000000 |  Iterations: 4
    0.8934   -1.7318   -0.0049    0.0000   -0.0000

pertubation size: 0.0001000000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8943   -1.7358    0.0726   -0.0006    0.0000         0

pertubation size: 0.0000100000 | root after perturbation: -0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0811   -0.0008    0.0000   -0.0000

pertubation size: 0.0000010000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0819   -0.0008    0.0000         0

pertubation size: 0.0000001000 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000         0

pertubation size: 0.0000000100 | root after perturbation: -0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000   -0.0000

pertubation size: 0.0000000010 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000         0

pertubation size: 0.0000000001 | root after perturbation: 0.000000 |  Iterations: 5
    0.8944   -1.7362    0.0820   -0.0008    0.0000    0.0000
```

Figure 15: Secant Method

```
Secant Method Testing
Test Interval 1:     -3    -1

Root: -1.994323 | Iterations: 10
   -3.0000   -1.0000   -1.3333  -28.0000   -1.3370   -1.3406   -3.4855   -1.5426   -1.6997   -2.1851   -1.9557   -1.9943

Test Interval 2:     -1     1

Root: 0.000000 | Iterations: 1
    -1     1     0

Test Interval 3:      1     3

Root: 2.000000 | Iterations: 10
    1.0000    3.0000    1.3333    1.6082    2.5145    1.8622    1.9568    2.0050    1.9998    2.0000    2.0000    2.0000
```

Figure 16: Regula Falsi

```
Regula Falsi Testing
Test Interval 1:     -3    -1

Regula Falsi Root: -1.998851 | Iterations: 10
   -3.0000   -1.3333   -1.6082   -1.7914   -1.8957   -1.9496   -1.9761   -1.9888   -1.9947   -1.9975   -1.9989

Test Interval 2:     -1     1

Regula Falsi Root: 0.000000 | Iterations: 1
    -1     0

Test Interval 3:      1     3

Regula Falsi Root: 1.998851 | Iterations: 10
    1.0000    1.3333    1.6082    1.7914    1.8957    1.9496    1.9761    1.9888    1.9947    1.9975    1.9989

Test Interval 4:     -3     1

Unuseable interval
Regula Falsi Root: 2.000000 | Iterations: 1
    -3     2
```

5.2.2 2.2: Scaling Three Distinct Roots

Building of Task 2.1 the next sets of test are concerned with what happens as $f(x) \to \sigma f(x)$ where $\sigma > 0 \in \mathbb{Z}^+$. During testing all Methods were unaffected by the scalar except for Steffensen's Method which suffers quite heavily. Because Steffensen's is approximating the derivative using the difference equation, when scaled the difference becomes larger causing instability in the iterations. This causes the method to update to far ahead and sometimes locate the non targeted root. Unlike Steffensen's, all the other method were invariant to the effect of the scaling and preformed as normally expected. For Newton and Secant the scaling acts as a linear transformation on both the rise and run of the slope.

Figure 17: Scaled Newton and Steffensen

```
Current sigma: 5
Scaled Newtons behavior for x > rho
root found: 2.000000 |  Iterations: 7
Scaled Newtons Iteration Progress
    4.0000    2.9091    2.3021    2.0507    2.0018    2.0000    2.0000    2.0000

Scaled Steffencen behavior for x > rho
root found: 3.933778 |  Iterations: 100
Scaled Newtons Iteration Progress
  Columns 1 through 24

    4.0000    3.9992    3.9984    3.9976    3.9968    3.9960    3.9952    3.9944    3.9936    3.9928    3.9920    3.9912    3.9904    3.9896    3.9888    3.9880    3.9872    3.9864    3.9856    3.9848    3.9840    3.9832    3.9824    3.9816

  Columns 25 through 48

    3.9808    3.9800    3.9792    3.9784    3.9776    3.9768    3.9760    3.9752    3.9743    3.9735    3.9727    3.9719    3.9711    3.9703    3.9695    3.9686    3.9678    3.9670    3.9662    3.9654    3.9646    3.9637    3.9629    3.9621

  Columns 49 through 72

    3.9613    3.9605    3.9596    3.9588    3.9580    3.9572    3.9564    3.9555    3.9547    3.9539    3.9531    3.9522    3.9514    3.9506    3.9497    3.9489    3.9481    3.9473    3.9464    3.9456    3.9448    3.9439    3.9431    3.9423

  Columns 73 through 96

    3.9414    3.9406    3.9397    3.9389    3.9381    3.9372    3.9364    3.9355    3.9347    3.9339    3.9330    3.9322    3.9313    3.9305    3.9297    3.9288    3.9280    3.9271    3.9263    3.9254    3.9246    3.9237    3.9229    3.9220

  Columns 97 through 101

    3.9212    3.9203    3.9195    3.9186    3.9178
```

Figure 18: Scaled Secant and Regula Falsi

```
Current sigma: 5
Scaled Secant behavior for x0 > rho and x1 > rho
root found: 2.000000 |  Iterations: 9
Scaled Secant Iteration Progress
    3.0000    4.0000    2.5455    2.3253    2.0883    2.0173    2.0011    2.0000    2.0000    2.0000    2.0000

Scaled RF behavior for x0 > rho and x1 > rho
Unuseable interval
root found: 2.010593 |  Iterations: 10
Scaled RF Iteration Progress
    3.0000    2.5455    2.3253    2.2028    2.1296    2.0841    2.0551    2.0363    2.0240    2.0159    2.0106
```

5.2.3 2.3: Root Coalescing

Consider the function $f(x) = x(x - \rho_1)(x - \rho_2)$ where $0 < \rho_1 < \rho_2$. as $\rho_1 \to 0$, Newton's Method slows down immensely to the point where the iteration is barely moving at all. This becomes even worse as $\rho_2 \to 0$ as well, causing even smaller steps than before. This is due to the fact that as the values of ρ get closer to zero the multiplicity of the root is numerically becoming unstable as it rises, like noticed earlier in Task 1. The Secant Method was much more effective in evaluate the roots while maintaining a strong super-linear convergence rate as $\rho_1, \rho_2 \to 0$. The secant method was able to locate roots at 0. The secant method also located the root at 10 when ρ_2 was stationary and ρ_1 was approaching zero.

Figure 19: Coalescing Root

```
 (NEWTONS) Coalescing Roots as rho1 -> 0 and Initial Guess: 1.000000
Rho1: 5.000000 |  Iterations: 1000000 | Root: 0.999843
Rho1: 2.000000 |  Iterations: 1000000 | Root: 1.000893
Rho1: 0.100000 |  Iterations: 1000000 | Root: 0.999950
Rho1: 0.010000 |  Iterations: 1000000 | Root: 0.999947
Rho1: 0.001000 |  Iterations: 1000000 | Root: 0.999947
Rho1: 0.000100 |  Iterations: 1000000 | Root: 0.999947
Rho1: 0.000010 |  Iterations: 1000000 | Root: 0.999947

 (NEWTONS) Coalescing Roots as rho1 & rho2 -> 0
Rho1: 5.000000 |  Rho2: 10.000000 |  Iterations: 1000000 | Root: 0.999843
Rho1: 2.000000 |  Rho2: 4.000000 |  Iterations: 1000000 | Root: 1.000300
Rho1: 0.100000 |  Rho2: 0.200000 |  Iterations: 1000000 | Root: 0.999970
Rho1: 0.010000 |  Rho2: 0.020000 |  Iterations: 1000000 | Root: 0.999967
Rho1: 0.001000 |  Rho2: 0.002000 |  Iterations: 1000000 | Root: 0.999967
Rho1: 0.000100 |  Rho2: 0.000200 |  Iterations: 1000000 | Root: 0.999967
Rho1: 0.000010 |  Rho2: 0.000020 |  Iterations: 1000000 | Root: 0.999967

 (SECANT) Coalescing Roots as rho1 -> 0
Rho1: 5.000000 |  Iterations: 7 | Root: 0.000000
Rho1: 2.000000 |  Iterations: 8 | Root: -0.000000
Rho1: 0.100000 |  Iterations: 13 | Root: 0.100000
Rho1: 0.010000 |  Iterations: 17 | Root: 0.010000
Rho1: 0.001000 |  Iterations: 21 | Root: -0.000000
Rho1: 0.000100 |  Iterations: 8 | Root: 10.000000
Rho1: 0.000010 |  Iterations: 7 | Root: 10.000000

 (SECANT) Coalescing Roots as rho1 & rho2 -> 0
Rho1: 5.000000 |  Rho2: 10.000000 |  Iterations: 7 | Root: 0.000000
Rho1: 2.000000 |  Rho2: 4.000000 |  Iterations: 8 | Root: -0.000000
Rho1: 0.100000 |  Rho2: 0.200000 |  Iterations: 10 | Root: 0.200000
Rho1: 0.010000 |  Rho2: 0.020000 |  Iterations: 10 | Root: 0.020000
Rho1: 0.001000 |  Rho2: 0.002000 |  Iterations: 10 | Root: 0.002000
Rho1: 0.000100 |  Rho2: 0.000200 |  Iterations: 9 | Root: 0.000200
Rho1: 0.000010 |  Rho2: 0.000020 |  Iterations: 5 | Root: 0.000021
```

5.3 Task 3: Nonlinear Relaxation Iterations

The following tasks deviate from earlier methods and consider a different approach to solving nonlinear equations. Consider the function $f(x) = x^2 - x - 2$ which has two roots at $\alpha_1 = -1$ and $\alpha_2 = 2$. Then define the following iterations:

$$\phi_1 = x^2 - 2 \quad \phi_2 = \sqrt{2+x} \quad \phi_3 = -\sqrt{2+x} \tag{8}$$

The question and it's analysis are from problem 6.3 in the study questions. Each iteration is distinct with their own unique characteristics. The First Iteration is the most unreliable of the three. This is due to the fact that even though it can be defined as a fixed point, each root acts as a repellent numerically. This is further evidence by $|\phi(\alpha)| > 1$ for α_1, α_2 and thus iterations within the neighbor of the iteration repel against the root.

Figure 20: Iteration 1 with Initial Guess = -1.5

```
 Iteration 1
Iteration: 1 | x = 0.250000
Iteration: 2 | x = -1.937500
Iteration: 3 | x = 1.753906
Iteration: 4 | x = 1.076187
Iteration: 5 | x = -0.841821
Iteration: 6 | x = -1.291337
Iteration: 7 | x = -0.332449
Iteration: 8 | x = -1.889478
Iteration: 9 | x = 1.570126
Iteration: 10 | x = 0.465297
Iteration: 11 | x = -1.783499
Iteration: 12 | x = 1.180869
Iteration: 13 | x = -0.605549
Iteration: 14 | x = -1.633311
Iteration: 15 | x = 0.667703
Iteration: 16 | x = -1.554172
Iteration: 17 | x = 0.415451
Iteration: 18 | x = -1.827400
Iteration: 19 | x = 1.339391
Iteration: 20 | x = -0.206031
Iteration: 21 | x = -1.957551
Iteration: 22 | x = 1.832006
Iteration: 23 | x = 1.356247
Iteration: 24 | x = -0.160593
Iteration: 25 | x = -1.974210
```

Evaluating ϕ_2 next, we are able to conclude from the solution analysis that $\phi(\alpha_1)$ is not a fixed point and will therefore not converge to α_1. Looking at the other root, $\phi(\alpha_2)$ is a fixed point that leads to a contraction mapping. This from looking at the derivative $\frac{1}{2\sqrt{2+x}}$ and seeing that happens as $x \to \infty$. This forces the value of derivative smaller with each step forcing the contraction mapping to α_2. Testing confirms that for $x^0 < -2$ the iteration is undefined, $x^{(0)} > -7/4$ converges to α_2 via the contraction mapping, and $-2 \leq x^{(0)} < -7/4$ converges to α_2 by moving into the region of contraction within one iteration.

Figure 21: Iteration 2 with Initial Guess = 10

```
 Iteration 2
Iteration: 1 | x = 3.464102
Iteration: 2 | x = 2.337542
Iteration: 3 | x = 2.082677
Iteration: 4 | x = 2.020563
Iteration: 5 | x = 2.005134
Iteration: 6 | x = 2.001283
Iteration: 7 | x = 2.000321
Iteration: 8 | x = 2.000080
Iteration: 9 | x = 2.000020
Iteration: 10 | x = 2.000005
Iteration: 11 | x = 2.000001
Iteration: 12 | x = 2.000000
Iteration: 13 | x = 2.000000
Iteration: 14 | x = 2.000000
Iteration: 15 | x = 2.000000
Iteration: 16 | x = 2.000000
Iteration: 17 | x = 2.000000
Iteration: 18 | x = 2.000000
Iteration: 19 | x = 2.000000
Iteration: 20 | x = 2.000000
Iteration: 21 | x = 2.000000
Iteration: 22 | x = 2.000000
Iteration: 23 | x = 2.000000
Iteration: 24 | x = 2.000000
Iteration: 25 | x = 2.000000
```

The last iteration to consider is ϕ_3 where α_1 is a fixed point via $\phi_3(\alpha_1) = \alpha_1$ while α_2 is not. To keep this iteration real the bounds $-2 < x < 2$ must be satisfied as well. Then looking at the derivative $-\frac{1}{2\sqrt{2+x}}$ where we can derive if $x \in (-7/4, 2) \rightarrow |\phi_3'| < 1$ and therefore points with the closed interval $[-7/4, 2]$ converge to α_1. The main point to notice here is that the sign dictates the root that iteration converges too and is further evidence through the simulation results.

Figure 22: Iteration 3 with Initial Guess = 0

```
Iteration 3
Iteration: 1 | x = -1.414214
Iteration: 2 | x = -0.765367
Iteration: 3 | x = -1.111140
Iteration: 4 | x = -0.942793
Iteration: 5 | x = -1.028205
Iteration: 6 | x = -0.985796
Iteration: 7 | x = -1.007077
Iteration: 8 | x = -0.996455
Iteration: 9 | x = -1.001771
Iteration: 10 | x = -0.999114
Iteration: 11 | x = -1.000443
Iteration: 12 | x = -0.999779
Iteration: 13 | x = -1.000111
Iteration: 14 | x = -0.999945
Iteration: 15 | x = -1.000028
Iteration: 16 | x = -0.999986
Iteration: 17 | x = -1.000007
Iteration: 18 | x = -0.999997
Iteration: 19 | x = -1.000002
Iteration: 20 | x = -0.999999
Iteration: 21 | x = -1.000000
Iteration: 22 | x = -1.000000
Iteration: 23 | x = -1.000000
Iteration: 24 | x = -1.000000
Iteration: 25 | x = -1.000000
```

5.4 Task 4: Nonlinear Relaxation Iterations for Systems

The last section of testing and evaluating nonlinear solvers uses questions 6.10(b) and 6.10(c) to confirm the convergence analysis from the associated solution. Consider the system of equations:

$$x^2 + y^2 = 4 \quad \text{and} \quad e^x + y = 1 \tag{9}$$

With the associated functions and iterations for G_1 and G_2:

$$G_1 = \begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} \gamma_1(x_k, y_k) \\ \gamma_2(x_k, y_k) \end{pmatrix} = \begin{pmatrix} \ln(1 - y_k) \\ -\sqrt{4 - x_k^2} \end{pmatrix} \tag{10}$$

$$G_2 = \begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} \gamma_1(x_k, y_k) \\ \gamma_2(x_k, y_k) \end{pmatrix} = \begin{pmatrix} -\sqrt{4 - y^2} \\ 1 - e^{x_k} \end{pmatrix} \tag{11}$$

And their respective Jacobian:

$$J_{G_1} = \begin{pmatrix} 0 & \frac{-1}{1-y} \\ \frac{x}{\sqrt{4-x^2}} & 0 \end{pmatrix} \quad \text{and } J_{G_2} = \begin{pmatrix} 0 & \frac{y}{\sqrt{4-y^2}} \\ -e^x & 0 \end{pmatrix} \tag{12}$$

For convergence, the spectral radius $\rho(J_G) < 1$. For G_1 the solution analysis shows that for $0 < x < \sqrt{2}$ and $-2 < y < 0$, the partial derivative are bounded to create a contraction mapping towards the right root. The left root does not meet this requirement and this acts as a repelling fixed point. Looking similarly at G_2, the domain is bounded by $-2 < x < 0$ and $0 < y < \sqrt{2}$ forcing a contraction mapping for the left most root. Analysis of the Jacobian also indicates that the right root is a repellent fixed point as well.

The results from the testing confirm the results that initial point not within the specified interval will not converge. For initial guesses that start within the correct intervals result in a right root at $x_r \approx 1.002$ and $y_r \approx -1.729$ and a left root at $x_l \approx -1.816$ and $y_l \approx 0.837$.

Figure 23: G_1 Iterations

```
 Initial Points: , x0 = 1.314214, y0 = -0.100000
Iteration: 1 | x = 0.095310 | y = -1.507595
Iteration: 2 | x = 0.919324 | y = -1.997728
Iteration: 3 | x = 1.097855 | y = -1.776188
Iteration: 4 | x = 1.021079 | y = -1.671740
Iteration: 5 | x = 0.982730 | y = -1.719709
Iteration: 6 | x = 1.000525 | y = -1.741907
Iteration: 7 | x = 1.008654 | y = -1.731748
Iteration: 8 | x = 1.004942 | y = -1.727026
Iteration: 9 | x = 1.003211 | y = -1.729188
Iteration: 10 | x = 1.004004 | y = -1.730193
Iteration: 11 | x = 1.004372 | y = -1.729733
Iteration: 12 | x = 1.004204 | y = -1.729519
Iteration: 13 | x = 1.004125 | y = -1.729617
Iteration: 14 | x = 1.004161 | y = -1.729662
Iteration: 15 | x = 1.004178 | y = -1.729642
Iteration: 16 | x = 1.004170 | y = -1.729632
Iteration: 17 | x = 1.004167 | y = -1.729636
Iteration: 18 | x = 1.004168 | y = -1.729638
Iteration: 19 | x = 1.004169 | y = -1.729637
Iteration: 20 | x = 1.004169 | y = -1.729637

 Initial Points: , x0 = 2.000000, y0 = 1.000000
Left real valued domain at iteration 1
```

Figure 24: G_2 Iterations

```
 Initial Points: , x0 = -1.990000, y0 = 1.990000
Iteration: 1 | x = -0.199750 | y = 0.863305
Iteration: 2 | x = -1.804080 | y = 0.181064
Iteration: 3 | x = -1.991787 | y = 0.835374
Iteration: 4 | x = -1.817182 | y = 0.863549
Iteration: 5 | x = -1.803963 | y = 0.837517
Iteration: 6 | x = -1.816195 | y = 0.835355
Iteration: 7 | x = -1.817191 | y = 0.837357
Iteration: 8 | x = -1.816269 | y = 0.837518
Iteration: 9 | x = -1.816195 | y = 0.837369
Iteration: 10 | x = -1.816264 | y = 0.837357
Iteration: 11 | x = -1.816269 | y = 0.837368
Iteration: 12 | x = -1.816264 | y = 0.837369
Iteration: 13 | x = -1.816264 | y = 0.837368
Iteration: 14 | x = -1.816264 | y = 0.837368
Iteration: 15 | x = -1.816264 | y = 0.837368
Iteration: 16 | x = -1.816264 | y = 0.837368
Iteration: 17 | x = -1.816264 | y = 0.837368
Iteration: 18 | x = -1.816264 | y = 0.837368
Iteration: 19 | x = -1.816264 | y = 0.837368
Iteration: 20 | x = -1.816264 | y = 0.837368

 Initial Points: , x0 = -3.000000, y0 = 3.000000
Left real valued domain at iteration 1
```

5.5 Conclusion

As seen from the collected evidence above, iterative methods and their implementations are effective nonlinear solvers that have many different moving parts. Geometry and mathematical analysis are the basis for most iterative methods and have been shown to be true when analyzed properly and with care. Having a deep understanding of the mathematical analysis ensure the proper mechanics behind the methods and simulation that have been run have helped to evidence this claim. Iterative methods and other non linear relaxation methods are a powerful mathematical tools that must be properly analyzed and implemented. This study highlights that a successful nonlinear analysis requires not just the implementation of an algorithm, but a prior mathematical characterization of the function's derivative behavior and stability regions.

5.6 Program Files

This report includes 10 Matlab files. The first 4 files include the iterative methods as function, while the rest were used for testing. Each file has been named according to method or the assigned task at hand within the report. Included

within the report are only a select few examples, while actual driver every interval tested during validation.